VICTRON VENUS OS · CAMPER VAN

Tank sensors & fridge for Venus OS

Step-by-step guide: wiring, ESP32 firmware upload, tank calibration and installation on a Raspberry Pi running Venus OS.

Mopeka Check Standard / XL — LPG bottle level

ESP32-S3 hydrostatic sender — fresh-water level for non-square tanks, water temperature

Bluetooth compressor fridge — fridge and freezer temperature (Alpicool controller, e.g. Hyckes)

github.com/mich-crypto/Venusosmods · generated 2026-10-29

Contents

Installation guide

- Part 1 – Prepare Venus OS
- Part 2 – Build and wire the ESP32 water sensor
- Part 3 – Upload the firmware to the ESP32
- Part 4 – Calibrate the water tank
- Part 5 – Install the drivers on Venus OS
- Part 6 – Mopeka gas sensor
- Part 7 – Fridge (Hyckes and other Bluetooth compressor fridges)
- Part 8 – Settings page in the browser
- Part 9 – Updates, troubleshooting, removal

Wiring

- Parts
- Power
- Pressure sensor, option C: 0–3.3 V output (e.g. QDY30A 0–3.3 V)
- Pressure sensor, option A: voltage output (0.5–4.5 V, 5 V supply)
- Pressure sensor, option B: 4–20 mA two-wire transmitter (12–24 V supply)
- DS18B20 temperature probe (optional)
- Choosing and mounting the pressure sensor
- ADC pins
- Junction box layout

Calibrating the water tank

- 0. Before you start
- 1. Zero point (both methods)
- 2A. Calibrate by filling in steps
- 2B. Calculate it from the tank's dimensions
- 3. Check and fine-tune
- Command reference

Filling chart: wheel-arch tank

- How the tank was modelled
- Where the sensor goes
- The table

Checking it

Adjusting

Before you start: this is a do-it-yourself project, not a Victron product. Work on the 12 V system with the fuse removed, and keep drinking-water parts food-safe. The steps have been tested in simulation. Each part of the guide ends with a check, so confirm it on your own installation before moving on.

Installation guide

This guide takes you from parts on the table to tanks and the fridge on your Venus OS screen and in VRM. Work through it in order. Each part ends with a check, so you know it works before moving on.

Part	What	Time
1	Prepare Venus OS (root access, SSH, Bluetooth)	15 min
2	Build and wire the ESP32 water sensor	1–2 h
3	Upload the firmware to the ESP32	20 min
4	Calibrate the water tank	30 min
5	Install the drivers on Venus OS	20 min
6	Mopeka gas sensor	10 min
7	Fridge	10 min
8	Settings page in the browser	5 min
9	Updates, troubleshooting, removal	–

You need: the Raspberry Pi running Venus OS on the same network (Wi-Fi or LAN) as a laptop, a USB-C **data** cable (some cables only charge), and the parts from wiring.md.

Part 1 – Prepare Venus OS

1. Open the Remote Console: on the Pi's screen, or in a browser at `http://venus.local` (or the Pi's IP address).
2. **Enable superuser access.** In *Settings* → *General*:
    1. Set *Access level* to *User & Installer* (password `ZZZ`).
    2. Highlight *Access level* without opening it, then press and hold the **right arrow** key for 5 seconds. *Superuser* now appears.
    3. Choose *Superuser*, then *Set root password*.
    4. Turn on *SSH on LAN*.

    Menus differ slightly between Venus OS versions. Victron's "Venus OS: Root Access" documentation has the details for each.
3. **Bluetooth:** *Settings* → *Bluetooth* → enabled.
4. **Check:** on the laptop, open a terminal (Windows: PowerShell) and run:

    ```
    ssh root@venus.local
    ```

    If `venus.local` isn't found, use the IP address from *Settings* → *Ethernet / Wi-Fi*. You should get a `root@raspberrypi...:~#` prompt. Type `exit` to leave.

Part 2 – Build and wire the ESP32 water sensor

Follow wiring.md. In short:

1. Set the buck converter to **5.0 V** with a multimeter *before* connecting anything to it.
2. Wire buck → diode → ESP32 `5V`, plus GND.
3. Connect the pressure sensor (option C: 0–3.3 V like the QDY30A, A: 0.5–4.5 V, or B: 4–20 mA) to GPIO4 through the resistors.
4. Optional: DS18B20 to GPIO5 with the 4.7 kΩ pull-up to 3V3.
5. Don't put the sensor in the tank yet; it's easier to test on the table.

Check: with the 12 V on, the ESP32's power LED is on and the buck converter still reads 5.0 V (about 4.7 V after the diode).

Part 3 – Upload the firmware to the ESP32

3.1 Install the Arduino IDE (once)

1. Download **Arduino IDE 2** from arduino.cc and install it.
2. *File → Preferences → Additional boards manager URLs*, add:

   ```
   https://espressif.github.io/arduino-esp32/package_esp32_index.json
   ```

3. *Tools → Board → Boards Manager*: search **esp32**, install **esp32 by Espressif Systems** (3.x; 2.0.17 works too).
4. *Tools → Manage Libraries*: install **OneWire** (Paul Stoffregen) and **DallasTemperature** (Miles Burton). Only needed with the DS18B20.

3.2 Open and configure the sketch

1. Download this repository (GitHub: *Code → Download ZIP*) and unzip it.
2. *File → Open →* `firmware/esp32_hydro_tank/esp32_hydro_tank.ino`.
3. In the **CONFIGURATION** block at the top, set:
   - `INPUT_CURRENT_LOOP` and the `SIGNAL_*` values for your sensor. The defaults are for the QDY30A 0–3.3 V, 2 m range (option C). The values for the other options are in wiring.md.
   - `SENSOR_RANGE_MM`: the sensor's full scale in mm of water. From the datasheet: 1 m H_2O = 1000 mm, 10 kPa ≈ 1020 mm, 0.1 bar ≈ 1020 mm.
   - `USE_DS18B20`: `1` with the probe, `0` without
   - `DEVICE_ID`: leave at `1` unless you build a second sender
   - `DEFAULT_TABLE`: leave as it is. You calibrate later over USB, without editing code.

3.3 Board settings (*Tools* menu)

Setting	Value
Board	**ESP32S3 Dev Module**
USB CDC On Boot	**Enabled** (otherwise you see no serial output)
Flash Size	match your board (4 MB / 8 MB / 16 MB; 4 MB always works)
Partition Scheme	Default
Upload Speed	921600 (use 115200 if uploads fail)
Port	the new COM / `/dev/ttyACM*` / `/dev/cu.usbmodem*` port

Many S3 boards have **two** USB-C ports, labelled *USB* and *UART* (or *COM*). Either works for uploading. With *USB CDC On Boot: Enabled* the Serial Monitor works on the **USB** port. On the *UART* port, set *USB CDC On Boot* to *Disabled*.

3.4 Upload

1. Unplug the buck converter's 5 V if you didn't fit the diode.
2. Connect the ESP32 by USB, choose the port, click **Upload** (→).
3. If it hangs at `Connecting....`: hold the **BOOT** button, tap **RST**, release BOOT, and upload again. After uploading, tap **RST** once.

3.5 Check

Tools → Serial Monitor, **115200 baud**, line ending **Both NL & CR**. You should see:

```
Using the default tank table from the sketch (no calibration stored)
Advertising as VHT, MAC 34:85:18:xx:xx:xx
Type 'help' for calibration commands.
signal=0.512 height=3.0mm litres=0.4 level=0.5% temp=19.6 activity=0.4
```

- `signal` should be close to `SIGNAL_AT_ZERO` with the sensor in air (0 V for the QDY30A 0–3.3 V, 0.5 V or 4 mA for the other types).
- Dip the sensor into a bucket: `height` should rise by roughly the depth.
- `FAULT` = signal outside the fault limits: check wiring and the `SIGNAL_*` settings.
- `temp=nan` = no DS18B20 found (wiring, pull-up, or `USE_DS18B20 0`).

Note the **MAC** for later (optional in config.ini).

Part 4 – Calibrate the water tank

Your wheel-arch tank: the firmware already contains its filling chart, calculated from the technical drawing (tank-filling-chart.md). Put the sensor at the bottom of the tank's leg, park level, empty the tank and type `zero`. Then check with a few buckets as described in that chapter.

For any other tank, or to measure yours exactly, follow calibration.md. Short version, with the van level and the tank empty:

```
zero
cal start
   (pour 10 L in, wait 30 s)
cal add 10
   (repeat until full)
cal save
```

No measuring container? Use the dimension calculator in calibration.md, method B.

Part 5 – Install the drivers on Venus OS

5.1 Copy the files

From an iPad or phone (Termius or another SSH app), no computer needed: log in to the Pi and let it download the files from GitHub itself:

```
cd /tmp
wget -O vom.tar.gz https://github.com/mich-crypto/Venusosmods/archive/refs/heads/master.tar.gz
tar -xzf vom.tar.gz
cp -r Venusosmods-master/data/shared_drivers Venusosmods-master/data/etc /data/
rm -rf vom.tar.gz Venusosmods-master
```

The Pi needs internet access for this (it has it if VRM works). Then continue with 5.2.

From a computer, in the folder where you unzipped the repository:

macOS / Linux / Windows PowerShell:

```
scp -r data/shared_drivers data/etc root@venus.local:/data/
```

On Windows you can also use **WinSCP** (SCP protocol, user `root`): copy the folders `data/shared_drivers` and `data/etc` into `/data/` on the Pi.

5.2 Create the configuration

```
ssh root@venus.local
sh /data/shared_drivers/install.sh
```

The first run creates `/data/shared_drivers/config.ini` and stops.

5.3 Find your sensors

```
python3 /data/shared_drivers/diagnostics/test_venus_compatibility.py --scan 30
```

Press the Mopeka's sync button during the scan. The scan lists:

- `PASS Mopeka Standard AA:BB:...` → the gas sensor is received. You don't need to note the MAC: the driver pairs with the sync button (Part 6)
- `PASS ESP32 tank id 1 ...` → the water sensor is received (MAC optional)

- `PASS Fridge candidate ...` → see Part 7

The upper part of the output checks the environment (Python, D-Bus, velib_python, Bluetooth, config). Fix any **FAIL** before continuing.

5.4 Edit config.ini

```
nano /data/shared_drivers/config.ini
```

(`Ctrl+O`, `Enter` saves, `Ctrl+X` quits.) Set the fridge MAC, the bottle size and `propane_ratio` for the Mopeka, and names. Every option is explained in the file. Delete or ignore sections for sensors you don't have.

5.5 Enable the services

Only the ones you use:

```
sh /data/shared_drivers/install.sh mopeka_standard esp32_hydro fridge
```

This runs the diagnostics first and refuses to enable anything if one reports FAIL. It links the services into `/service` and adds them to `/data/rc.local` so they come back after reboots and Venus OS updates.

5.6 Check

- Remote Console: the **tanks** appear on the overview page and under *Device list*. The water temperature and fridge zones appear as temperature sensors in the *Device list*.
- Logs:
  ```
  tail -F /var/log/esp32_hydro/current | tai64nlocal
  ```
- Live D-Bus values: `dbus-spy` (arrow keys, `Enter` opens a service, `q` quits).
- **VRM:** after a few minutes, the tanks and temperatures appear on the VRM dashboard and in *Advanced* widgets. To rename a device, use its *Device* page on the GX; VRM picks the new name up.
- Reboot the Pi once (`reboot`) and check everything comes back.

Part 6 – Mopeka gas sensor

Pairing (no MAC needed): leave `mac =` empty in config.ini. After enabling the service, press the **sync button** on the Mopeka once, with the bottle within Bluetooth range of the Pi. The log shows `Paired with Mopeka AA:BB:...`, and the tank appears. The MAC is stored in Venus OS settings, so it survives reboots and updates. Other Mopeka sensors nearby (e.g. at a campsite) are ignored.

```
tail -F /var/log/mopeka_standard/current | tai64nlocal    # watch it pair
```

To pair a different sensor later:

```
dbus -y com.victronenergy.settings /Settings/Devices/mopeka_standard/Mac SetValue ''
svc -t /service/mopeka_standard          # then press sync on the new sensor
```

1. Put the sensor on the **centre** of the bottle's bottom, with plenty of coupling gel (or the supplied pad).
2. In `dbus-spy`, open `com.victronenergy.tank.mopeka_standard` and watch `/Mopeka/DistanceMm` and `/Mopeka/EchoQuality`.
    - EchoQuality stays low (<3) or the distance jumps around: re-seat the sensor, use more gel, clean the bottle bottom.
3. Note the distance with a nearly **empty** bottle → `empty_mm`, and with a freshly **filled** bottle → `full_mm`. Gas bottles are filled to about 80 %, so "full" is what a new bottle reads.
4. `propane_ratio`: 1.0 for pure propane (most of Northern Europe, and winter gas), around 0.3–0.4 for summer propane/butane mixes (e.g. southern Europe). This corrects the speed of sound in the liquid.
5. `capacity_l`: liquid volume, e.g. 11 kg ≈ 27 L, 5 kg ≈ 12 L.
6. To make the level match the Mopeka app at this moment, open the settings page → *Gas*, type the percentage the app shows next to *Right now the Mopeka app says* and press *Match it and save*. The page works out "Reading when full" from the current sensor reading and your bottle curve and saves it. Redo it right after a refill for the best accuracy. `svc -t /service/mopeka_standard` restarts the driver after changes.
7. **Sensor battery:** the *Gas* tab shows the sensor's battery charge and voltage, a graph of the voltage (recorded every 10 minutes) and a "low" mark when it falls to 20 %. With Telegram alerts switched on you get a message once (and again after 12 hours if it stays low). Change the limit with *Gas sensor battery alert at* in the Telegram settings.
8. The sensor's temperature is also published as its own temperature sensor, `com.victronenergy.temperature.mopeka_standard_temp` ("Gas bottle temperature", type outdoor), so it shows up with the other environment sensors in the *Device list* and on VRM. Turn it off with `temperature_service = no`, or change the type with `temperature_type` (battery, fridge, generic, room, outdoor, waterheater, freezer).

Part 7 – Fridge (Hyckes and other Bluetooth compressor fridges)

Many 12 V compressor fridges use the same Bluetooth controller, the one the "Alpicool / CAR FRIDGE FREEZER" app talks to. The driver reads the temperatures of both zones, the set points, the supply voltage and the eco/power state. It does **not** change any fridge settings.

Hyckes fridges use this controller (confirmed with a dual-zone Hyckes). For another brand, or to check yours, test it first:

1. **Close the fridge app on your phone**, or switch off the phone's Bluetooth. The fridge accepts only one connection at a time.
2. On the Pi:

    ```
    python3 /data/shared_drivers/diagnostics/fridge_probe.py
    ```

    Look for `→ LIKELY FRIDGE` (names like `WT-0001`, `A1-...`, `AK1-...`). No candidate? Note the devices with the strongest signal while standing next to the fridge.
3. Test it:

    ```
    python3 /data/shared_drivers/diagnostics/fridge_probe.py AA:BB:CC:DD:EE:FF
    ```

    - `OK - this fridge speaks the Alpicool protocol` plus temperatures: put the MAC in `[fridge]` in config.ini and enable it with `install.sh fridge`.

- `characteristics ... not found`: Hyckes uses a different controller. Keep the printed *GATT characteristics* list: with it (and ideally a Bluetooth capture from the phone app), the protocol module can be extended.

Sharing with the phone app: by default the Pi connects for about 2 s every 60 s (`poll_interval`) and then disconnects, so the phone app can still connect in between. While the phone app is connected, the Pi can't read the fridge; after `timeout` seconds the zones show as disconnected and recover automatically.

Where the fridge shows up in the Remote Console

- **Temperatures:** each zone is a temperature sensor ("Fridge", "Freezer"). They appear with the other environment sensors (on the *Levels* page's environment view in the newer GUI) and in the *Device list*.
- **Controls:** the *switch pane* (the switches icon in the top bar) shows a "Fridge" card with *Fridge power*, *Fridge eco mode* and a set point control per zone. A change is sent right away: the Pi connects, sends the command, reads the status back, and the control shows what the fridge actually accepted. Set points stay within the range the fridge itself reports (e.g. 0…8 °C for the fridge, −20…−12 °C for the freezer).
- Set point ranges follow the manual of the dual-zone fridge: fridge 0…8 °C, freezer −20…−12 °C (a single-compartment model: −20…20 °C). If yours differs, set `zone1_min`, `zone1_max`, `zone2_min`, `zone2_max` in `[fridge]`.
- The controls are their own device, `com.victronenergy.switch.fridge_controls` (the pane only lists switch devices). It appears after the first successful read of the fridge, so give it a minute. It needs a Venus OS version that has the switch pane (v3.5x or newer with the new GUI). If the icon is missing, check the *Device list* for "Compressor fridge controls".
- **Controls on the settings page:** the *Fridge* tab has the same controls (power, eco mode, a slider with − and + per zone, limited to the range the fridge allows). It shows "The fridge confirmed." when the fridge has taken the change. They use the same path as the switch pane, so they work only when the controls device exists and the phone app is not connected to the fridge.
- Don't want controls in Venus OS? Set `control = no` in the `[fridge]` section.

Part 8 – Settings page in the browser

Instead of editing `config.ini` in nano, you get a web page on the Pi. It shows the water tank, gas bottle and fridge live, and it changes the settings of the fridge, gas, water and the battery driver (dbus-serialbattery, for example the JK BMS) with proper fields, units and explanations. It is made for a phone or iPad.

Turn it on (Termius is fine):

```
sh /data/shared_drivers/update.sh
sh /data/shared_drivers/install.sh web_ui
```

The installer prints a random **password** and the address, for example `http://192.168.1.50:8088`. The password is also in `config.ini` under `[web]` if you forget it. Open the address in Safari and add it to the home screen.

What is on the page

- **Overview** – live values and the state of every driver (running, restarts, last log line). Anyone on your network can look; nobody can change anything without the password.

- **Fridge, Gas, Water** – one form per sensor. Bottle presets fill in volume and shape, *Use as empty* / *Use as full* copy the current sensor reading, a blank field means "use the default". Press *Save*, or *Save & restart driver* to apply it at once.
- **Battery** – three views for dbus-serialbattery (for example a JK BMS):
  - *Live & cells*: charge ring, voltage, current, power, temperatures, what the BMS allows, and a plain-language "What I notice" list (cells far apart, one cell always lowest, temperatures, current limits, alarms). Bar chart of every cell, graphs of current, charge, voltage, cell spread, every cell over time, each cell's difference from the average, temperatures and the current the BMS allows. Choose 15 min, 1 hour or 6 hours; touch or hover a graph for the exact values, tap a cell in the legend to hide it. The Pi records a point every 10 s while the settings page runs and keeps 6 hours in memory. Cell voltages need `BATTERY_CELL_DATA_FORMAT = 1` (the default).
  - *Limits & curves*: how the BMS is connected, the cell voltage limits as a picture (with the pack voltage for your cell count and where your cells are now), the main numbers, one graph per current limit curve (charge and discharge against cell voltage, temperature, MOSFET temperature and charge level, with a marker where you are now) and a table of everything you changed from the defaults.
  - *All settings*: every setting in groups, with search, switches, the default value, a *Reset to default* link and the original explanation under *About this setting*. Only the settings you change are written to its `config.ini`; the file with the defaults is never touched. Changes to Bluetooth BMS, the USB Bluetooth option or the CAN port need a reboot of the Pi (the page offers a *Reboot now* button).
- **Starlink** – modelled on the open-source Stardashy dashboard, with these views:
  - *Dashboard*: the last speed test (download, upload, latency and jitter, with the change since the previous test, bufferbloat grade and the Cloudflare server it used), your public IP, ping tiles with a history strip for Cloudflare, Google, Quad9 and the dish itself (add the Starlink gateway `100.64.0.1` to tell a dish problem from an internet problem), charts of the tests, and the data used in the billing cycle with an optional cap bar.
  - *Dish*: live downlink, uplink and PoP latency, alerts, obstruction, GPS, tilt, firmware, graphs and a statistics grid. A "throttled" badge shows when the dish reports that your speed is limited.
  - *Sky*: where the dish points (azimuth, elevation, tilt). Tracking which satellite serves you needs orbit data and a large calculation and is not included.
  - *Energy*: the dish's power draw now, today, this month and this year in kWh, a projection for the month, the cost if you set a price, and a bar chart per day. Counted from the dish's own per-second readings.
  - *Tests*: every speed test, failed ones included (never counted as 0 Mbps).
  - *Log*: availability, an outage timeline (red = the dish reported an outage and why, grey = the Pi could not reach the dish), the incident table and the history of your public IP address.
  - *Debug*: ping, traceroute, DNS lookup and an HTTP check run on the Pi.
  - *Settings*: **software update** (compares with GitHub and runs `update.sh`), the Starlink settings (speed test interval, ping targets, billing day, data cap, electricity price) and **Telegram alerts** (dish offline and back, packet loss, hardware alerts, new public IP, failed speed tests; make a bot with @BotFather). **In Venus OS:** the dish also appears as a device called *Starlink connection* (Device list, Levels page and VRM). There is no Starlink device type, so it is shown as a tank-type device whose *level* is the connection quality from 0 to 100 %: 85 and up excellent, 65 good, 40 fair, below that poor, 0 offline. The score starts at 100 and loses points for packet loss, blocked sky, high ping, a speed limit, a weak signal and dish alerts (an outage is 0). Speed, ping, loss and obstruction are available on D-Bus under `/Starlink/...`. Switch

it off with *Show the dish in Venus OS* in the Starlink settings. It disappears when the dish has been unreachable for a minute. The Pi must reach the dish at `192.168.100.1` (check: `curl http://192.168.100.1/`). Speed tests are off until you press the button or set an interval: each moves about 33 MB, so 30 minutes is about 1.6 GB a day. Energy, data use, outages and test results are kept in `/data/shared_drivers/state/starlink.json`. Starlink does not publish the interface and this was not tested against a real dish by the author: if a view shows an error run `python3 /data/shared_drivers/diagnostics/starlink_probe.py` and send the output.

- **Raw data** – what the sensors really send, for debugging. *Gas* shows every Mopeka advertisement with its raw bytes (`data=`), the decoded distance, temperature, battery and echo quality, and the level that was published. *Fridge* shows what the Pi sends (`tx`) and receives (`rx`), the decoded status, and the last status payload byte by byte with the meaning of each byte (set points, ranges, temperatures, voltage). *Water* shows the ESP32 advertisements. The log lives in `/tmp/venusosmods-raw/` on the Pi, so it starts empty after a reboot and never wears the SD card. Use *Copy* to paste it into a message.
- **Files & logs** – the raw config files as a fallback, and the log of every service.

Each save keeps the previous file as `config.ini.bak` next to it.

Safety

- The page listens on the local network only: the installer opens port 8088 in the Pi's firewall for 192.168.x.x, 10.x.x.x, 172.16–31.x.x and Tailscale addresses, not for the internet. Don't forward the port on your router.
- From outside the van, use Tailscale or an SSH tunnel (`ssh -L 8088:localhost:8088 root@<pi>`, then open `http://localhost:8088`).
- Change the port, switch the firewall rule off (`open_firewall = no`) or bind to the Pi itself (`bind = 127.0.0.1`) in the `[web]` section.
- Page not reachable? `tail -F /var/log/web_ui/current | tai64nlocal` and `python3 /data/shared_drivers/diagnostics/test_venus_compatibility.py`.

Part 9 – Updates, troubleshooting, removal

Updating the drivers

Your settings are not touched by an update: `config.ini` is not in the download, and `update.sh` also keeps a copy as `config.ini.pre-update`. Saves from the settings page keep the previous version as `config.ini.bak`. On the Pi (from Termius too):

```
sh /data/shared_drivers/update.sh
```

It downloads the latest version from GitHub, copies it over the old files and restarts the running services. `config.ini` is not overwritten (it isn't in the repository).

Venus OS firmware updates

Nothing to do: everything lives in `/data`, and `/data/rc.local` restores the services at boot. Afterwards, run the diagnostics once to confirm.

Troubleshooting

Symptom	Check
Tank not shown	`svstat /service/<name>` should say `up`; read the log in `/var/log/<name>/current`
Service keeps restarting	run it by hand to see the error: `svc -d /service/<n>` then `python3 /data/shared_drivers/<driver>.py --section <n> --debug`
Tank shows "disconnected"	the sensor isn't heard: run `--scan 30`, check the range (metal lockers block Bluetooth)
Water level wrong	redo `zero` and the calibration; check the sensor lies flat and the vent tube is open
Water level slow while driving	intended (anti-slosh), settles once parked
Mopeka level jumps	coupling gel, sensor position, `median_window = 9`
Fridge disconnected	phone app connected? Bluetooth range? `fridge_probe.py <MAC>`
Two devices with the same instance	*Settings → VRM online portal → VRM device instances* (Venus OS 3.x), or `device_instance` in config.ini

Test without hardware: the `tests/` folder contains a simulation that runs all drivers against a fake Bluetooth stack (see the README).

Removing

```
sh /data/shared_drivers/uninstall.sh            # all services
sh /data/shared_drivers/uninstall.sh fridge     # just one
```

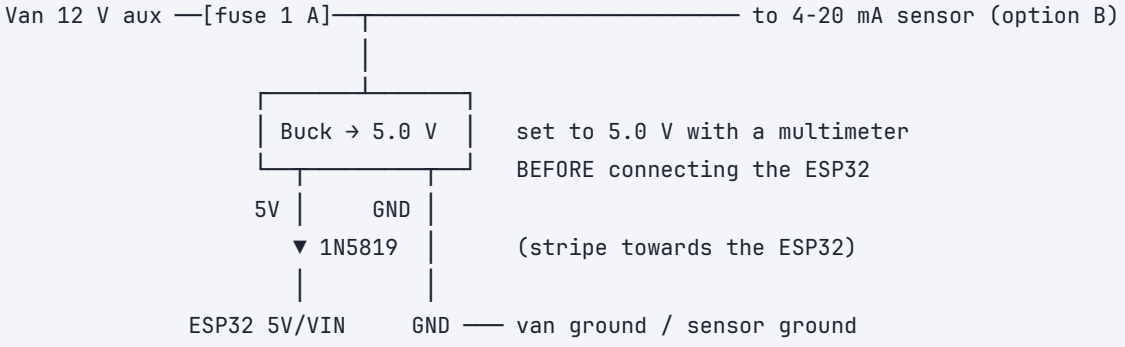
This stops the services and removes them from `/data/rc.local`. Delete `/data/shared_drivers` and `/data/etc/init.d/<name>` to remove the files.

Wiring

Parts

Part	Notes
ESP32-S3 dev board (DevKitC-1 or clone)	any S3 board with USB-C
Buck converter 12 V → 5 V, ≥1 A	MP1584 or LM2596 module, adjustable
Fuse holder + 1 A fuse	at the 12 V tap
Schottky diode 1N5819 / SS14	lets USB and 12 V be connected together
Hydrostatic level transmitter	0–3.3 V, 0.5–4.5 V or 4–20 mA output (see below)
Resistors	0–3.3 V version: 10 kΩ + 1 MΩ; 0.5–4.5 V version: 10 kΩ + 20 kΩ; 4–20 mA version: 120 Ω 0.1 %/1 %
1 kΩ resistor + 100 nF capacitor	input filter/protection (options A and B)
DS18B20 waterproof probe + 4.7 kΩ	optional water temperature
IP65 junction box, cable glands, ferrules	
Wire	0.75 mm² for 12 V, any thin wire for signals

Power



- Use the board's **5V / VIN** pin, never 3V3.
- The diode drops ~0.3 V (ESP32 sees ~4.7 V, fine) and stops the buck converter and the laptop's USB from feeding into each other while you calibrate. Without it: unplug the 5 V wire before plugging in USB.
- Take 12 V from a circuit that is on when the Victron system is on (e.g. the 12 V distribution after the main switch), not from the starter battery. The ESP32 draws about 0.5 W.

Pressure sensor, option C: 0–3.3 V output (e.g. QDY30A 0–3.3 V)

This is the firmware's default. The output can go straight to the ESP32, with no divider and no shunt resistor. The QDY30A datasheet is saved in datasheets/.

QDY30A 3-wire voltage version, colours per the datasheet:

```
red    (V+)   ── fused 12 V   (see "Supply voltage" below)
green  (V-)   ── GND  (common with the ESP32 GND)
yellow (out)  ──[10k]──┬──────┬── GPIO4
                       │      │
                    [100nF]  [1M]
                       │      │
                      GND    GND
```

Firmware: `INPUT_CURRENT_LOOP 0`, `DIVIDER_RATIO 1.0f`, `SIGNAL_AT_ZERO 0.0f`, `SIGNAL_AT_FULL 3.3f`, `SENSOR_RANGE_MM 2000.0f` (for the 2 m version), `SIGNAL_FAULT_LOW -1.0f`, `SIGNAL_FAULT_HIGH 3.05f`.

- **Supply voltage:** the datasheet lists 10–32 V DC for the voltage-output versions (0–5 V, 1–5 V) and 5 V only for the 0.5–4.5 V ratiometric version. The 0–3.3 V version is not in its table, and another seller lists a 0–3.3 V version for 5 V. **Ask the seller, or read the label on the cable, before powering it.** Most likely it is 10–32 V: then use the van's fused 12 V. If it says 5 V, use the ESP32's 5V pin instead.
- **Test before connecting the ESP32:** power the sensor alone and measure yellow to green with a multimeter: about 0 V in air, rising by about 0.165 V per 10 cm when you lower it into a bucket or a pipe filled with water.
- **Why 10 kΩ:** with 12 V on the sensor, a fault could push the output above 3.3 V. The 10 kΩ limits the current into the pin's protection diode to under 1 mA. With the 1 MΩ pull-down it reads about 1 % low, which the calibration cancels out.
- **Resolution:** the ESP32 reads up to about 3.1 V, i.e. about 1.88 m of water with the 2 m sensor. A van tank of 30–50 cm uses a quarter of the range: about 0.6 mm of water per mV, which the firmware's median filter handles well. The datasheet's ±2 % F.S. zero drift (±40 mm) is long-term drift: run `zero` on an empty tank once or twice a year.
- **No broken-wire detection:** 0 V means "empty" and also "wire broken". The 1 MΩ pull-down makes a broken signal wire read as empty instead of random values. Option B (4–20 mA) is the only one that can tell the two apart.
- **Vent tube:** the cable has a breather capillary (the datasheet's "breather capillary"). Keep the cable end with the tube inside the dry junction box, open to air.

Pressure sensor, option A: voltage output (0.5–4.5 V, 5 V supply)

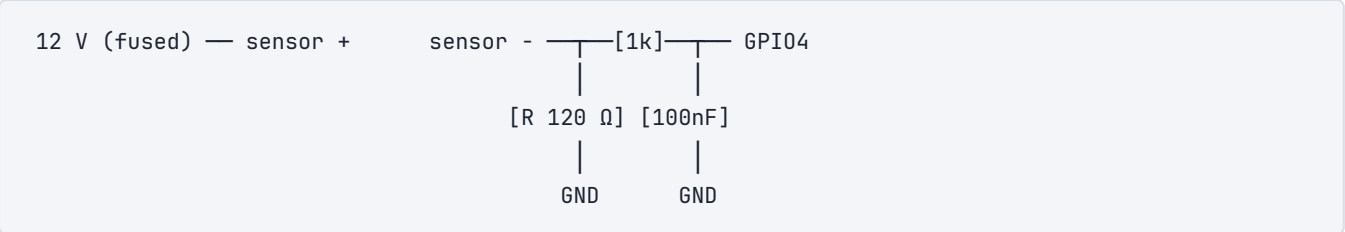
```
sensor red   (+5V) ── ESP32 5V
sensor black (GND) ── GND
sensor yellow(OUT) ──[R1 10k]──┬──[1k]──┬── GPIO4
                               │        │
                            [R2 20k] [100nF]
                               │        │
                              GND      GND
```

Divider ratio (R1+R2)/R2 = 1.5, so 4.5 V becomes 3.0 V at the pin. Firmware: `INPUT_CURRENT_LOOP 0`, `DIVIDER_RATIO 1.5f`, `SIGNAL_AT_ZERO 0.5f`, `SIGNAL_AT_FULL 4.5f`, `SIGNAL_FAULT_LOW 0.2f`,

`SIGNAL_FAULT_HIGH 4.9f`.

Wire colours differ between makers: check the sensor's label or datasheet.

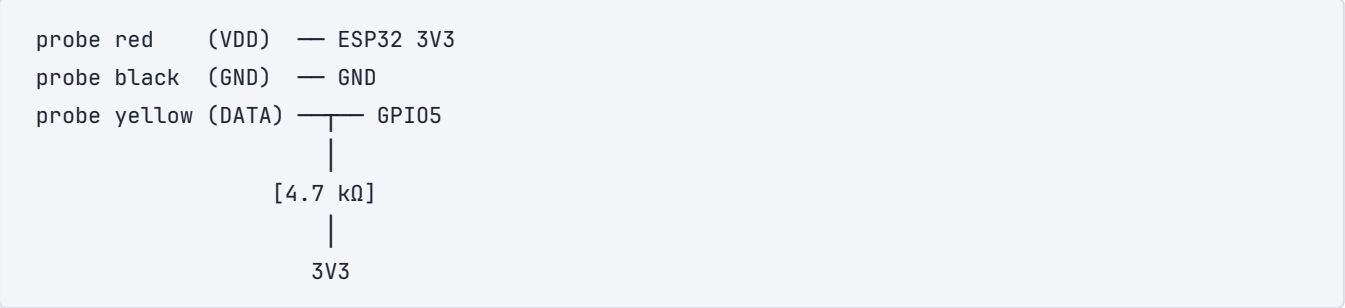
Pressure sensor, option B: 4–20 mA two-wire transmitter (12–24 V supply)



4 mA → 0.48 V, 20 mA → 2.4 V. Use a 0.1 % or 1 % resistor. Firmware: `INPUT_CURRENT_LOOP 1`, `SHUNT_OHMS 120.0f`, `SIGNAL_AT_ZERO 4.0f`, `SIGNAL_AT_FULL 20.0f`, `SIGNAL_FAULT_LOW 3.0f`, `SIGNAL_FAULT_HIGH 22.0f`. A broken wire then shows up as a fault in the GUI instead of "empty". This type copes best with long cables and electrical noise in a vehicle.

The 1 kΩ + 100 nF form an RC filter and limit current into the pin's protection diodes if something goes wrong.

DS18B20 temperature probe (optional)



- Power it from **3V3**, not 5 V: the data line goes straight to the ESP32.
- The 4.7 kΩ pull-up is required. For cables over ~5 m use 2.2 kΩ.
- Set `USE_DS18B20 0` in the sketch if you don't fit one.
- Drinking water: the common "waterproof stainless" probes are not certified for potable water. Either buy one with a documented food-grade sleeve and cable, or put the probe in a dry pocket on the **outside** of the tank wall (clamped with thermal paste and insulated). That measures the water through the wall, a little slower, without touching it.

Choosing and mounting the pressure sensor

- **Range:** pick one just above the tank height, e.g. 0–1 m H_2O (0–10 kPa) for a 40 cm tank. A 0–5 m sensor would only use 8 % of its range, so you lose resolution. A 2 m sensor still works well (see option C).
- **Vented (gauge) sensor:** the cable has a small vent tube. Keep its end open to air and dry: inside the junction box is fine, outside under the van is not. Don't seal it with the cable gland.
- **Position:** flat on the bottom, as close to the **middle** of the tank as you can. On a slope the level at the centre hardly changes.
- **Drinking water:** stainless (316) body and a food-safe (e.g. FDA/KTW certified) cable, or mount the sensor outside the tank in a T-piece on the drain at the lowest point.

- **Through the lid:** most installs lower the probe through a cable gland (PG9/PG11) in the tank's inspection lid, so no new hole below the water line.

ADC pins

On the ESP32-S3 only GPIO1..GPIO10 are ADC1. ADC2 (GPIO11..20) shares hardware with the radio and can return errors while it is active, so keep the sensor on ADC1. GPIO4 and GPIO5 are the defaults in the sketch (`PIN_PRESSURE`, `PIN_ONEWIRE`).

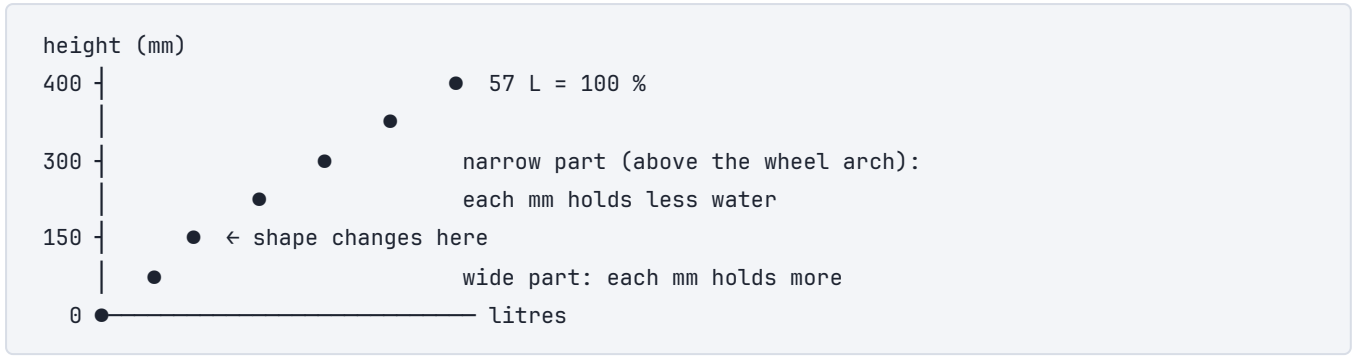
Junction box layout



Mount the box above the tank's maximum water level, with the glands pointing down so condensation drains away from the board. Plastic boxes let Bluetooth through; a metal box or a box inside a metal locker will cut the range.

Calibrating the water tank

A hydrostatic sensor measures **pressure**, which only depends on the **height** of the water above it, not on the tank's shape. In a square box, height and volume are proportional. In a van tank with a wheel-arch cut-out, a slope or rounded corners they are not: the first 5 cm may hold 12 L, the top 5 cm only 4 L. The ESP32 fixes this with a **height → litres table** that it interpolates between.



You do this once. The table is stored in the ESP32's flash and survives power cuts and firmware updates. (For the wheel-arch tank from the drawing, the table is already built in: see tank-filling-chart.md. You only need `zero` and a check.) There are two ways to build it:

- **A. Fill it in steps** (most accurate, ~30 minutes)
- **B. Calculate it from the tank's dimensions** (when you can't easily fill it in measured steps)

Both use the Arduino IDE **Serial Monitor** (115200 baud, "Both NL & CR") with the ESP32 connected to your laptop over USB-C. Only keep the van's 12 V connected at the same time if you fitted the Schottky diode from wiring.md; otherwise unplug the buck converter's 5 V first.

0. Before you start

1. Mount the sensor where it will stay, lying flat on the bottom, as close to the middle of the tank as possible.
2. Park the van **level** (use a spirit level on the floor, or an app).
3. Type `log` to stop the status lines scrolling, or leave them on to watch the height settle.
4. Type `status`: you should see a plausible `signal=` (0.5 V or 4 mA at zero). `FAULT` means a wiring problem; see wiring.md.

1. Zero point (both methods)

With the tank **completely empty** (or emptied down to the drain):

```
zero
```

The ESP32 averages the reading for 5 s and stores an offset so that the height now reads 0 mm. This removes the sensor's offset and the height of its membrane above the tank floor.

2A. Calibrate by filling in steps

You need a container of known size: a 10 L bucket, or a 5 L jerrycan. Use the same one every time, and measure what it really holds once, on a kitchen scale (1 L of water = 1 kg).

```
cal start
```

records 0 L at the current (empty) height. Then repeat:

1. Pour in one container (for example 10 L).
2. Wait **30 seconds** for the water to settle. If `SLOSH` is shown, wait longer.
3. Type the amount you added **in this step**:

   ```
   cal add 10
   ```

   The ESP32 measures for 5 s and stores the point with the running total (10, 20, 30 L...).

Use smaller steps (5 L) where the shape changes, e.g. around the top of the wheel arch. A wrong point can be removed with `cal undo`, and `cal show` prints the table so far.

When the tank is full:

```
cal save
```

The table is now active and stored. The last point is 100 %. `cal save` also prints the table as C code: copy it into `DEFAULT_TABLE` in the sketch as a backup (otherwise `reset`, or an ESP32 full flash erase, loses it).

Tip: fill through the normal filler neck, so the tank's breather and any air pockets behave like they will on the road.

2B. Calculate it from the tank's dimensions

Measure the **inside** of the tank (outside dimensions minus the wall, often 3-5 mm per wall for roto-moulded tanks) at every height where the shape changes:

- width (left–right) and depth (front–back) at that height
- if one of them is the same everywhere, you only need the other one

Example: a 300 mm deep tank that is 600 mm wide for the first 150 mm, then 400 mm wide (wheel arch) up to 400 mm:

```
python3 tools/tank_table.py --depth 300 --profile "0:600 150:600 151:400 400:400"
```

Width and depth both change? Use `height:width:depth`:

```
python3 tools/tank_table.py --profile "0:500:250 100:550:300 380:550:300"
```

Options:

- `--offset 15`: the sensor's membrane sits 15 mm above the floor, for example because of its housing. Water below it can't be measured, so the table starts counting litres from there.
- `--scale 0.97`: correct for internals (baffles, pump, fittings), or to match the volume the tank's datasheet gives.

The script prints `cal point ...` lines. Paste them into the Serial Monitor (after `zero`), followed by `cal save`.

Check: add a known amount (one bucket) and compare. If the reading is consistently off by a few %, re-run with `--scale`.

3. Check and fine-tune

- After calibration, fill and empty a few litres and watch `litres=` in the log. The level should change by what you added, within about 1-2 %.
- Driving: `SLOSH` appears and the reading becomes very slow (~25 s time constant). That is intended; it settles again within seconds when parked.
- Parked on a slope, the reading changes a little if the sensor isn't in the middle of the tank. That is the water level really changing at the sensor; only moving the sensor fixes it.
- Freezing: a hydrostatic sensor can be damaged when the water around it freezes. Drain the tank in frost, or use the van's tank heater.

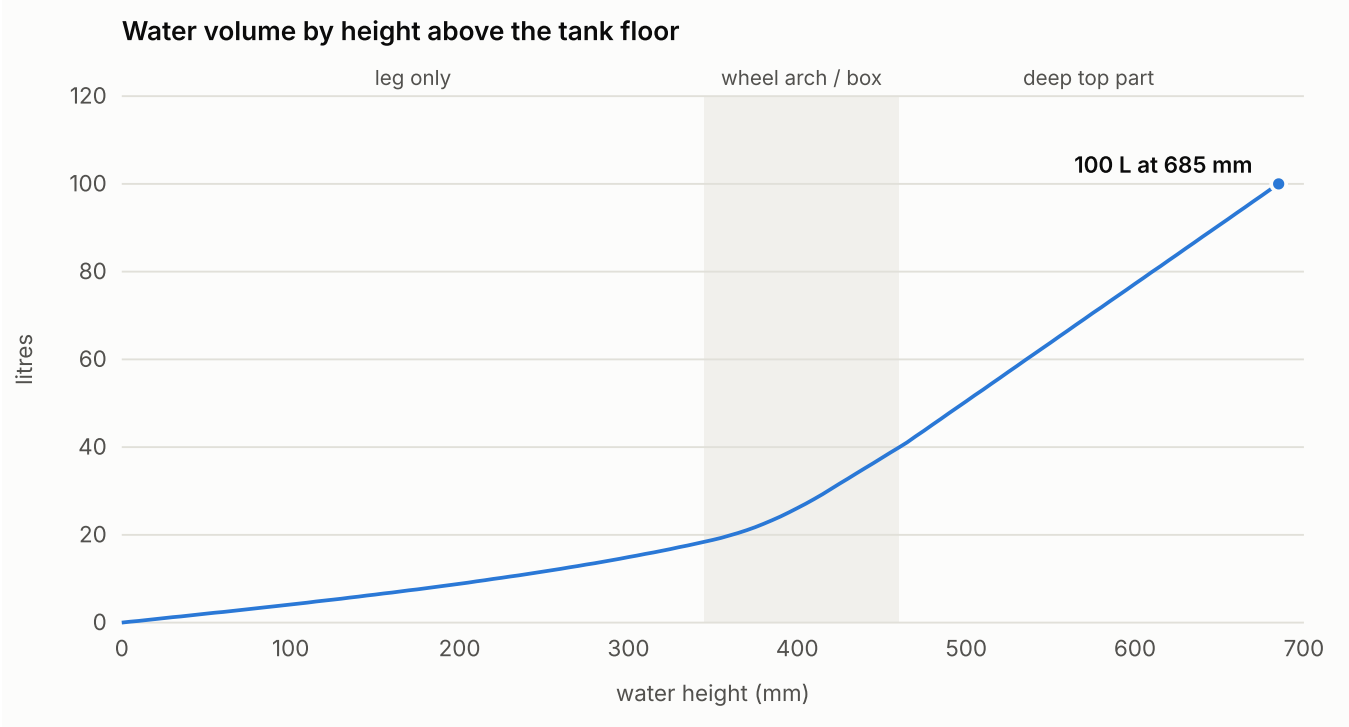
Command reference

Command	Does
`help`	list commands
`status`	signal, raw and filtered height, litres, active table
`log`	turn the periodic status line on/off
`zero`	empty tank: current height becomes 0 mm (stored)
`zero <mm>`	set the offset by hand
`cal start`	new table, 0 L at the current height
`cal add <L>`	record a point after adding `<L>` litres
`cal point <mm> <L>`	add a point by hand (total litres)
`cal undo` / `cal show` / `cal cancel`	edit the table being measured
`cal save`	activate + store the table, print it as code
`table`	print the active table as code
`reset`	erase stored calibration, use `DEFAULT_TABLE` from the sketch

The Venus OS side needs no changes after calibration: the ESP32 sends level %, litres and capacity, and the tank's capacity on the GX updates automatically.

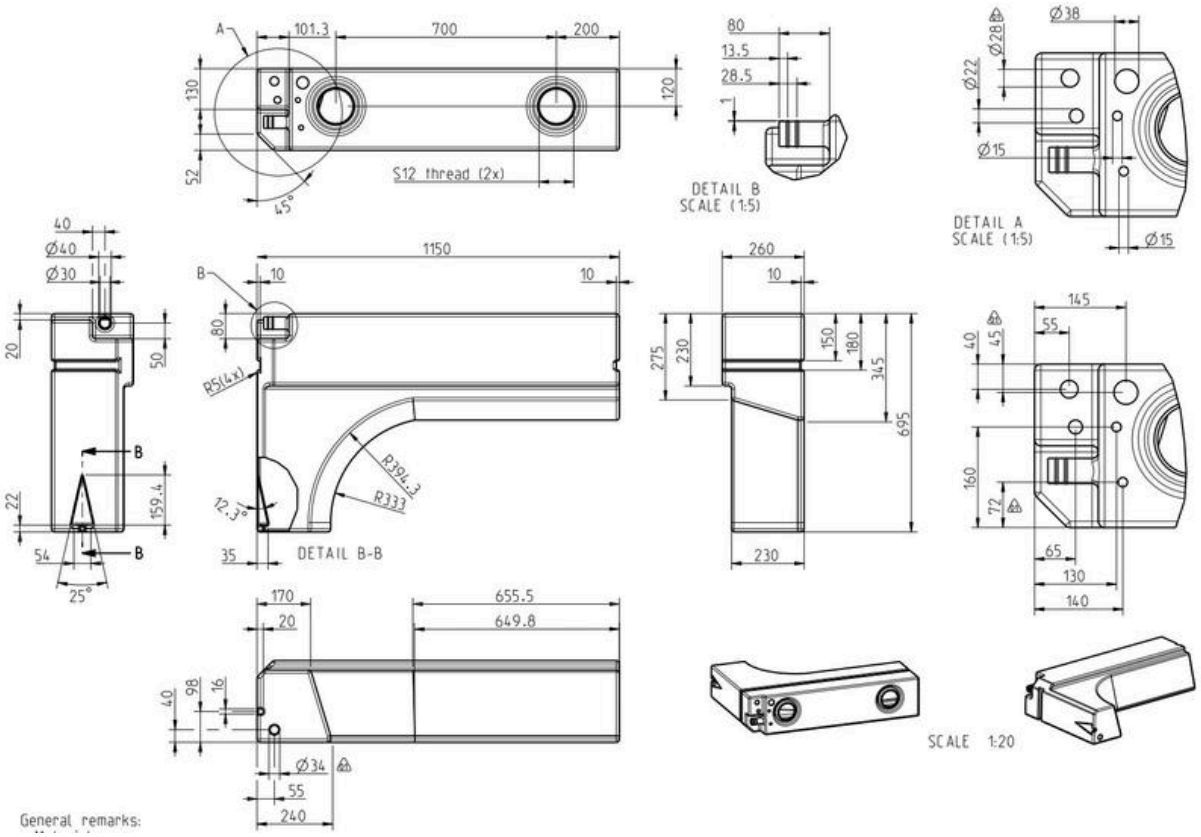
Filling chart: wheel-arch tank

Calculated from the tank's technical drawing (tank/tank-drawing.jpg) with `tools/tank_wheelarch.py`. **This table is already the firmware's default**, so after `zero` on the empty tank the ESP32 shows litres without any filling calibration.



Capacity: 100 L, the tank's nominal volume. The geometry alone gives 106 L (115 L from the outside dimensions, minus 5 mm walls), so the whole curve is scaled by 0.94. That covers thicker walls, internal parts, and the air space that stays under the top when the tank is filled to the brim of the filler. Full (100 %) is at 685 mm water height.

How the tank was modelled



Part	Dimensions used (outside, mm)
Total height / length	695 / 1150
Depth	260 in the top 230 mm, 230 below
Wheel arch	circle centred on the floor at 590 from the leg end; radius 420 on one face and 350 on the other (the arch is slanted across the depth)
Leg at the floor	170 mm long on one face, 240 on the other (590 − 420 and 590 − 350, matching the bottom view)
Arch top / box bottom	275 and 345 mm below the top (end view)
Walls	5 mm (typical for roto-moulded PE)

The drawing labels the arch curves R394.3 and R333. Those are the curved wall surfaces, which aren't centred on the floor. The radius used here (420 / 350 around a centre on the floor) is the one that matches the 170 / 240 mm leg and the 275 / 345 mm box bottom at the same time. The difference in volume is below 1 L.

The volume is added up in 1 mm height slices, and each slice follows the arch across the depth. Small details are ignored: the fittings, the V-shaped sump under the outlet, rounded edges (R5) and the 10 mm chamfers. Together they change the total by roughly 1–2 %.

Where the sensor goes

The leg is the only part that reaches the floor, so the sensor must lie **at the bottom of the leg**. Lower it through the threaded port above the leg (the S12 port 101 mm from the leg end in the top view). The heights in the table are measured from the leg's inner floor.

Parking on a slope matters more for this tank than for a box. The upper part is 1150 mm long, so when the van stands nose-up or nose-down, the water shifts along it. Above about 350 mm (around 20 L) the reading is only exact on level ground. The ESP32's slosh filter doesn't correct for a slope; park level when you want a precise number.

The table

Height (mm)	Litres	%
0	0.0	0
20	0.8	1
40	1.6	2
60	2.4	2
80	3.3	3
100	4.1	4
120	5.0	5
140	5.9	6
160	6.9	7
180	7.8	8
200	8.9	9
220	9.9	10
240	11.1	11
260	12.3	12
280	13.6	14
300	14.9	15
320	16.4	16
340	18.0	18
350	18.9	19
360	19.9	20
380	22.5	23
400	26.1	26
420	30.5	30
440	35.2	35
460	39.9	40
465	41.1	41
480	45.1	45
500	50.4	50
520	55.8	56
540	61.2	61
560	66.5	67
580	71.9	72

Height (mm)	Litres	%
600	77.2	77
620	82.6	83
640	87.9	88
660	93.3	93
680	98.7	99
685	100.0	100

Checking it

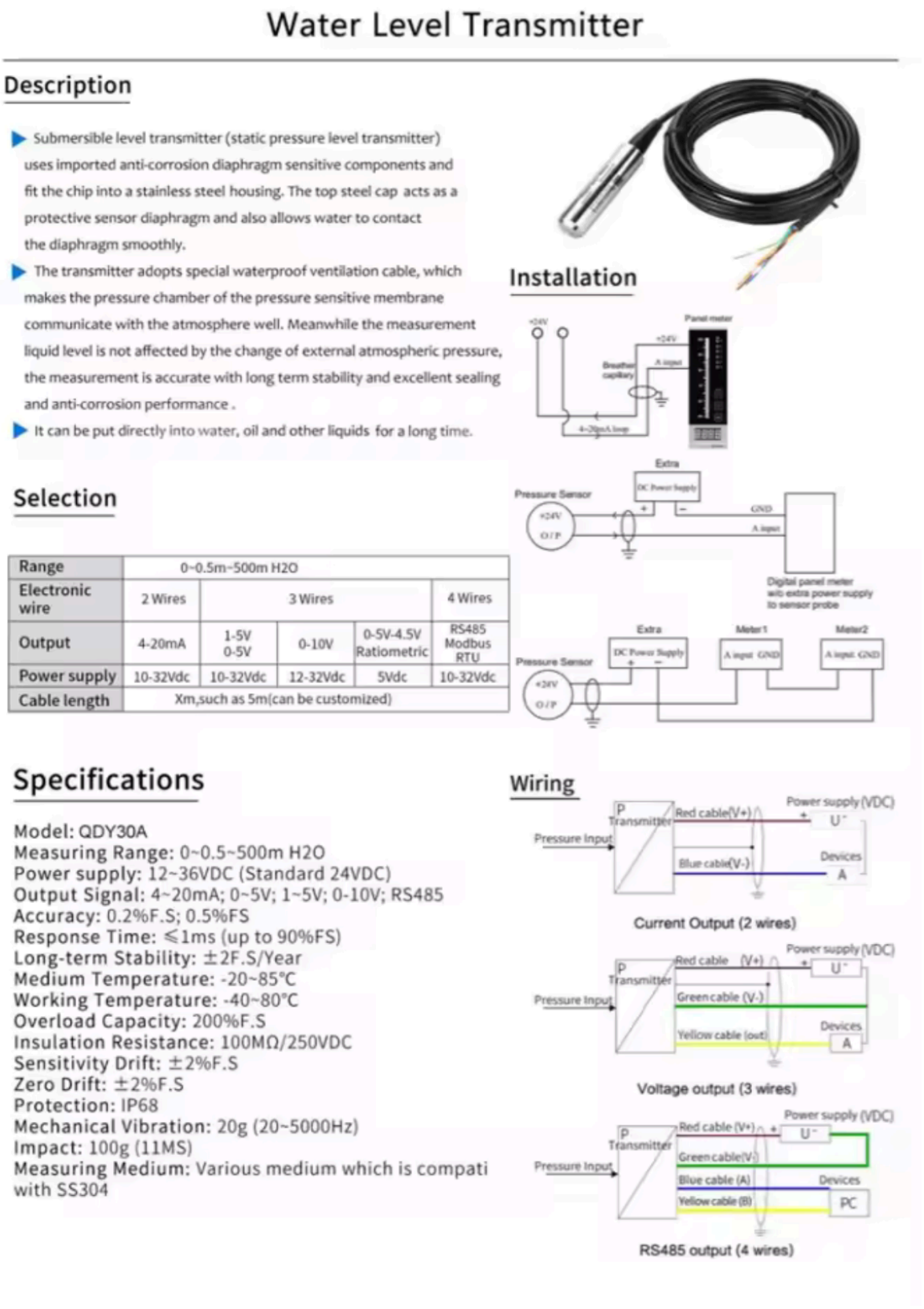
1. Empty the tank, park level, type `zero` in the Serial Monitor.
2. Pour in 10 L and compare `litres=` in the log with the table. Repeat a few times. The bottom part (leg) is the most sensitive to the model, and the top part the least.
3. Best check: fill the empty tank completely and note how much went in (water meter, or count the buckets). If that isn't 100 L, rescale to the real number (below). If one region is off (e.g. around the arch), calibrate by filling in steps instead (calibration.md, method A), which measures your real tank.

Adjusting

```
python3 tools/tank_wheelarch.py                 # the default: scaled to 100 L
python3 tools/tank_wheelarch.py --capacity 95   # e.g. if it really holds 95 L
python3 tools/tank_wheelarch.py --capacity 0    # pure geometry (106 L)
```

Each prints `cal point ...` lines to paste into the Serial Monitor (followed by `cal save`), plus the C table for `DEFAULT_TABLE` in the sketch. `--markdown --svg docs/tank/tank-filling-chart.svg` regenerates this table and the chart.

Appendix: datasheets



Water Level Transmitter

Description

- Submersible level transmitter (static pressure level transmitter) uses imported anti-corrosion diaphragm sensitive components and fit the chip into a stainless steel housing. The top steel cap acts as a protective sensor diaphragm and also allows water to contact the diaphragm smoothly.
- The transmitter adopts special waterproof ventilation cable, which makes the pressure chamber of the pressure sensitive membrane communicate with the atmosphere well. Meanwhile the measurement liquid level is not affected by the change of external atmospheric pressure, the measurement is accurate with long term stability and excellent sealing and anti-corrosion performance .
- It can be put directly into water, oil and other liquids for a long time.

Installation

Selection

Range	0~0.5m~500m H2O				
Electronic wire	2 Wires	3 Wires			4 Wires
Output	4-20mA	1-5V 0-5V	0-10V	0-5V-4.5V Ratiometric	RS485 Modbus RTU
Power supply	10-32Vdc	10-32Vdc	12-32Vdc	5Vdc	10-32Vdc
Cable length	Xm,such as 5m(can be customized)				

Specifications

Model: QDY30A
Measuring Range: 0~0.5~500m H2O
Power supply: 12~36VDC (Standard 24VDC)
Output Signal: 4~20mA; 0~5V; 1~5V; 0-10V; RS485
Accuracy: 0.2%F.S; 0.5%FS
Response Time: ⩽1ms (up to 90%FS)
Long-term Stability: ±2F.S/Year
Medium Temperature: -20~85℃
Working Temperature: -40~80℃
Overload Capacity: 200%F.S
Insulation Resistance: 100MΩ/250VDC
Sensitivity Drift: ±2%F.S
Zero Drift: ±2%F.S
Protection: IP68
Mechanical Vibration: 20g (20~5000Hz)
Impact: 100g (11MS)
Measuring Medium: Various medium which is compati with SS304

Wiring

Current Output (2 wires)

Voltage output (3 wires)

RS485 output (4 wires)

QDY30A-water-level-transmitter.png